Custom Model Binding in ASP.NET Core Web API

In this article, I will discuss How to Implement **Custom Model Binding in ASP.NET Core Web API** Application with Examples. Please read our previous article discussing **Model Binding Using FromBody in ASP.NET Core Web API** with Examples.

Default Model Binder Limitations in ASP.NET Core Web API

The Default Model Binder in ASP.NET Core Web API efficiently binds data from HTTP requests (such as query strings, route values, form data, and body) to parameters and model objects in your controller actions. However, it has some limitations:

- **Limited to Built-in Sources:** The default model binder works out of the box with standard types (e.g., integers, strings, complex types, collections, etc.), but it doesn't handle custom, non-standard formats (e.g., a complex object represented as a custom string format).
- **Lacks Flexibility:** Complex logic required for binding from multiple parts of the request (e.g., headers, query, body) is difficult with the default model binder.
- **No Support for Special Data Types:** Unless specifically configured, the default binder can't handle particular data types like tuples, complex dictionaries, or non-standard collections.
- **Performance Issues for Large Data:** For large data transfers, default binding methods like FromBody (which typically reads the whole payload into memory) may cause performance bottlenecks.

Example to Understand the above Limitations:

Let's implement an ASP.NET Core Web API project demonstrating the limitations of the default model binder. In this example, we will use Many model classes and a Sample Controller to demonstrate. So, create a class named **Sample.cs** within the Models folder and then copy and paste the following code:

```
using System.ComponentModel.DataAnnotations;

namespace ModelBinding.Models
{
    //This Model will use to demonstrate Limited to Built-in Sources
    public class CustomObject
    {
        public string Name { get; set; }
        public int Age { get; set; }
```

```
        public string Location { get; set; }
    }

    //This Model will use to demonstrate Lacks Flexibility
    public class ComplexBodyModel
    {
        public string Data { get; set; }
    }

    //This Model will use to demonstrate Lacks Flexibility
    public class MergedModel
    {
        public string Header { get; set; }
        public string Query { get; set; }
        public string BodyData { get; set; }
    }

    //This Model will use to demonstrate No Support for Special Data Types
    public class CustomTupleModel
    {
        public int Item1 { get; set; }
        public string Item2 { get; set; }
    }

    //This Model will use to demonstrate Performance Issues for Large Data
    public class LargeDataModel
    {
        public List<string> LargeDataList { get; set; }
    }
}
```

Sample Controller:

Next, create an API Empty Controller named **SampleController** within the Controllers folder and copy and paste the following code. Here, I am creating different endpoints to show the various limitations of the Built-in Model binder.

```
using Microsoft.AspNetCore.Mvc;
using ModelBinding.Models;

namespace ModelBinding.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class SampleController : ControllerBase
    {
```

```
        //Limited to Built-in Sources
        //Limitation:
        //The model binder can't bind CustomObject directly from a custom string
format like "Name:Age:Location".
        [HttpGet("custom-object-binding")]
        public IActionResult CustomObjectBinding([FromQuery] string complexData)
        {
            // The data is in the custom format "Name:Age:Location"
            var parts = complexData?.Split(':');
            if (parts?.Length == 3)
            {
                var customObject = new CustomObject
                {
                    Name = parts[0],
                    Age = int.Parse(parts[1]),
                    Location = parts[2]
                };
                return Ok(customObject);
            }
            return BadRequest("Invalid custom format");
        }

        //Lacks Flexibility
        //Limitation:
        //The default model binder cannot easily handle merging data from
multiple sources(headers, query, and body) into a single model.
        [HttpPost("multi-source-binding")]
        public IActionResult MultiSourceBinding([FromHeader(Name = "X-Custom-
Header")] string headerValue,
        [FromQuery] string queryValue, [FromBody] ComplexBodyModel bodyModel)
        {
            // Merging data from header, query, and body
            var mergedResult = new MergedModel
            {
                Header = headerValue,
                Query = queryValue,
                BodyData = bodyModel?.Data
            };

            return Ok(mergedResult);
        }

        //No Support for Special Data Types
        //Limitation:
        //Tuples cannot be easily bound by the default model binder without
additional configuration or custom binding logic.
        [HttpPost("tuple-binding")]
```

```
        public IActionResult TupleBinding([FromBody] CustomTupleModel model)
        {
            return Ok($"Tuple Data: Item1 = {model.Item1}, Item2 =
{model.Item2}");
        }

        //Performance Issues for Large Data
        //Limitation:
        //For large payloads, FromBody reads the entire request body into memory,
which can be inefficient and lead to performance issues.
        [HttpPost("large-data-binding")]
        public IActionResult LargeDataBinding([FromBody] LargeDataModel model)
        {
            // Assume the data size is very large
            return Ok("Data received successfully");
        }
    }
}
```

Testing the Endpoints:

Limited to Built-in Sources (Custom Object Binding)

Endpoint: GET api/sample/custom-object-binding

Query Parameters:

complexData: Set this to a value like John:25:NYC.

Lacks Flexibility (Multi-Source Binding)

Endpoint: POST api/sample/multi-source-binding

Headers:

X-Custom-Header: Set this to HeaderValue.

Query Parameters:

queryValue: Set this to QueryValue.

Body (JSON):

{

“data”: “BodyValue”

}

No Support for Special Data Types (Tuple Binding)

Endpoint: POST api/sample/tuple-binding

Body (JSON):

```
{
  “Item1”: 123,
  “Item2”: “TestValue”
}
```

Performance Issues for Large Data (Large Data Binding)

Endpoint: POST api/sample/large-data-binding
Body (JSON):

```
{
  “largeDataList”: [
    “Data1”,
    “Data2”,
    “Data3”,
    “…”
  ]
}
```

Let us proceed and understand How to Implement Custom Model Bindings in ASP.NET Core Web Application to overcome the above Limitations:

What is Custom Model Binding in ASP.NET Core Web API?

Custom Model Binding in ASP.NET Core Web API allows developers to define the specialized logic of how the data from HTTP requests is mapped to action method parameters. This is particularly useful for scenarios where the default model binder can’t handle the data as needed. Custom model binders can read data from various parts of an HTTP request and convert it into .NET types. This involves implementing a custom binder class that can parse the incoming data (from any part of the HTTP request, such as headers, query strings, or body) and map it to the required .NET objects or parameters.

How to Implement Custom Model Binding in ASP.NET Core Web API?

To implement Custom Model Binding in ASP.NET Core Web API, we typically need to create a class that implements the **IModelBinder** interface and provides the implementation for the BindModelAsync method. BindModelAsync is the core method of the IModelBinder interface, and it is responsible for binding the incoming request data to a model. The ASP.NET Core framework calls it when an action method parameter or model requires binding.

The IModelBinder interface in ASP.NET Core defines custom model binding logic. A model binder maps incoming request data (e.g., from the URL, query string, form data, body, etc.) to action method parameters or properties of a

model in a controller. By default, the framework uses built-in model binders. You can implement custom model binders when you need to handle specific or complex scenarios that the default binders cannot manage.

Custom Model Binder for Complex Custom Object (Limited to Built-in Sources)

We will create a custom model binder that handles a complex object represented in a custom string format (**Name:Age:Location**). So, create a class file named **CustomObjectModelBinder.cs** within the Models folder and copy and paste the following code. The following class is to create a custom model binder in ASP.NET Core that binds an incoming string value from the request (Query String) to a custom model class (CustomObject). The string value is expected to be in a specific format, where properties are separated by colons (:). The custom binder parses the string, validates it, and binds it to the model class if the format is correct.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;

namespace ModelBinding.Models
{
    // Define a custom model binder by implementing the IModelBinder interface
    public class CustomObjectModelBinder : IModelBinder
    {
        // The BindModelAsync method is invoked by the ASP.NET Core model binding
system
        // to bind the incoming request data to the target model
        public Task BindModelAsync(ModelBindingContext bindingContext)
        {
            // Retrieve the value of the model binding field from the request
(e.g., from query string)
            var value =
bindingContext.ValueProvider.GetValue(bindingContext.FieldName).FirstValue;

            // Check if the value is null or empty, indicating a failed binding
attempt
            if (string.IsNullOrEmpty(value))
            {
                // Mark the binding attempt as failed
                bindingContext.Result = ModelBindingResult.Failed();

                // Return a completed task as no further processing is needed
                return Task.CompletedTask;
            }

            // Split the incoming string by colons to extract the individual
parts (Name, Age, Location)
            var parts = value.Split(':');
```

```
            // Check if the input string has exactly 3 parts (indicating correct
format)
            if (parts.Length == 3)
            {
                // Create a new instance of the CustomObject model and populate
it with the extracted values
                var customObject = new CustomObject
                {
                    Name = parts[0],             // First part is the Name
                    Age = int.Parse(parts[1]),   // Second part is the Age
(converted to integer)
                    Location = parts[2]          // Third part is the Location
                };

                // Mark the binding as successful and set the bound model
                bindingContext.Result = ModelBindingResult.Success(customObject);
            }
            else
            {
                // If the format is incorrect, mark the binding attempt as failed
                bindingContext.Result = ModelBindingResult.Failed();
            }

            // Return a completed task to indicate that the binding process has
finished
            return Task.CompletedTask;
        }
    }
}
```

Registering CustomObjectModelBinder with the CustomObject Class:

Please modify the CustomObject class as follows. The **[ModelBinder(BinderType = typeof(CustomObjectModelBinder))]** attribute is used to associate a custom model binder (CustomObjectModelBinder) with the CustomObject class. This tells the ASP.NET Core framework to use the specified custom model binder whenever it tries to bind data from the incoming request to the CustomObject class.

```
// The [ModelBinder] attribute is used to specify the custom model binder for
this class.
// The BinderType parameter is set to typeof(CustomObjectModelBinder), indicating
that the CustomObjectModelBinder
// should be used to bind incoming request data to instances of the CustomObject
class.
[ModelBinder(BinderType = typeof(CustomObjectModelBinder))]
public class CustomObject
{
```

```
    // This property will store the 'Name' part extracted by the custom model
binder
    public string Name { get; set; }

    // This property will store the 'Age' part extracted and parsed by the custom
model binder
    public int Age { get; set; }

    // This property will store the 'Location' part extracted by the custom model
binder
    public string Location { get; set; }
}
```

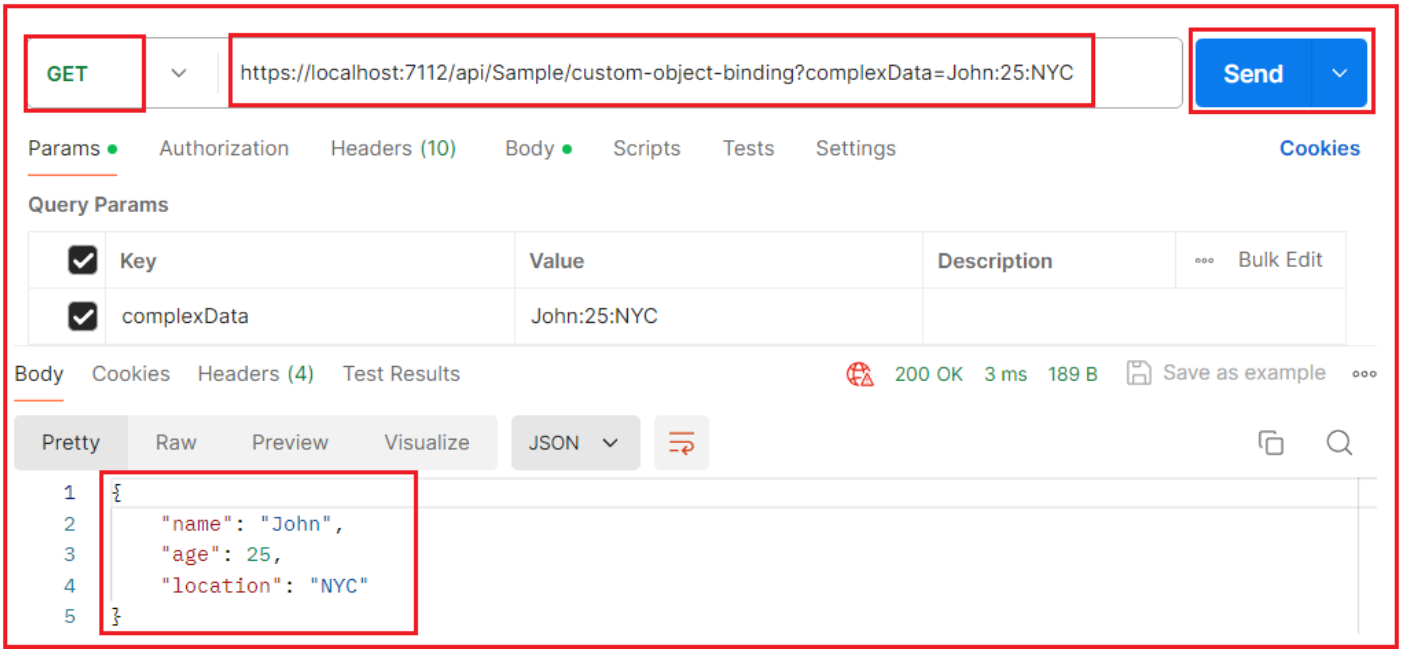
So, the **[ModelBinder(BinderType = typeof(CustomObjectModelBinder))]** annotation ensures that the custom logic provided in CustomObjectModelBinder is used to handle the binding of incoming data to the CustomObject model instead of relying on the default model binding mechanism.

Modifying the CustomObjectBinding Action Method:

Next, modify the CustomObjectBinding Action Method of the Sample Controller as follows:

```
[HttpGet("custom-object-binding")]
public IActionResult CustomObjectBinding(CustomObject complexData)
{
    return Ok(complexData);
}
```

So, with the above changes in place, access the above endpoint, and it should work as expected, as shown in the below image. I am using Postman to access the above endpoint.



Custom Model Binder for Multi-Source Binding (Lacks Flexibility)

Now, we will create a custom model binder that binds data from multiple sources: headers, query, and body. So, create a class file named MultiSourceModelBinder.cs within the Models folder and copy and paste the following code. Please Install the Newtonsoft.Json Package using the **Install–Package Newtonsoft.Json** command in the Package Manager Console. The following MultiSourceModelBinder class creates a custom model binder that aggregates data from multiple sources (headers, query strings, and request body) within an HTTP request.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;  // Importing ModelBinding namespace
to work with ASP.NET Core's model binding.
using Newtonsoft.Json;  // Importing Newtonsoft.Json for JSON deserialization.

namespace ModelBinding.Models
{
    // Custom model binder class implementing the IModelBinder interface to
provide custom logic for binding data to models.
    public class MultiSourceModelBinder : IModelBinder
    {
        // The core method to handle custom model binding.
        // This method gets called when the binding process is initiated.
        public async Task BindModelAsync(ModelBindingContext bindingContext)
        {
            try
            {
                // Accessing the HttpContext (HTTP request/response) for the
current model binding operation.
                var httpContext = bindingContext.HttpContext;

                // Error handling: Check if the request contains necessary data
sources like headers, query, and body.
                if (httpContext.Request.Headers == null ||
httpContext.Request.Query == null || httpContext.Request.Body == null)
                {
                    // If any of the sources are missing, add an error to the
ModelState and mark the binding as failed.

bindingContext.ModelState.AddModelError("MultiSourceModelBinder", "Missing
necessary data in the request.");
                    bindingContext.Result = ModelBindingResult.Failed();
                    return;  // Exit the method if any data source is missing.
                }

                // Fetching a custom header value ("X-Custom-Header") from the
HTTP request headers and converting it to a string.
                var headerValue = httpContext.Request.Headers["X-Custom-
Header"].ToString();
```

```
                // Fetching a value from the HTTP request query string
("queryValue") and converting it to a string.
                var queryValue =
httpContext.Request.Query["queryValue"].ToString();

                // Error handling: Validate header and query values to ensure
they are not empty or null.
                if (string.IsNullOrEmpty(headerValue))
                {
                    // Add an error to ModelState if the header value is missing
or empty.
                    bindingContext.ModelState.AddModelError("Header", "X-Custom-
Header is missing or empty.");
                    bindingContext.Result = ModelBindingResult.Failed();  // Mark
the binding as failed.
                    return;  // Exit the method if the header value is invalid.
                }

                if (string.IsNullOrEmpty(queryValue))
                {
                    // Add an error to ModelState if the query value is missing
or empty.
                    bindingContext.ModelState.AddModelError("Query", "Query
parameter 'queryValue' is missing or empty.");
                    bindingContext.Result = ModelBindingResult.Failed();  // Mark
the binding as failed.
                    return;  // Exit the method if the query value is invalid.
                }

                // Reading the entire body content of the HTTP request
asynchronously and converting it to a string.
                string bodyContent;
                try
                {
                    // Use a StreamReader to read the request body stream
asynchronously.
                    using (var reader = new
StreamReader(httpContext.Request.Body))
                    {
                        bodyContent = await reader.ReadToEndAsync();  // Read the
body content as a string.
                    }
                }
                catch (Exception ex)
                {
```

```
                    // If there is an error reading the body (e.g., stream
issues), add an error to ModelState.
                    bindingContext.ModelState.AddModelError("Body", $"Error
reading request body: {ex.Message}");
                    bindingContext.Result = ModelBindingResult.Failed();  // Mark
the binding as failed.
                    return;  // Exit the method if there is an error reading the
body.
                }

                // Deserializing the JSON string from the request body into a
ComplexBodyModel object.
                ComplexBodyModel bodyModel;
                try
                {
                    // Deserialize the JSON body content into the
ComplexBodyModel class.
                    bodyModel = JsonConvert.DeserializeObject<ComplexBodyModel>
(bodyContent);

                    // If the deserialization returns null, indicate a failure in
deserializing the body.
                    if (bodyModel == null)
                    {
                        bindingContext.ModelState.AddModelError("Body", "Failed
to deserialize the body content into ComplexBodyModel.");
                        bindingContext.Result = ModelBindingResult.Failed();  //
Mark the binding as failed.
                        return;  // Exit the method if deserialization fails.
                    }
                }
                catch (JsonException jsonEx)
                {
                    // If the JSON deserialization throws an exception (e.g.,
invalid JSON format), handle it here.
                    bindingContext.ModelState.AddModelError("Body", $"Invalid
JSON format in request body: {jsonEx.Message}");
                    bindingContext.Result = ModelBindingResult.Failed();  // Mark
the binding as failed.
                    return;  // Exit the method if deserialization fails.
                }

                // Creating a new instance of MergedModel and populating its
properties with data from header, query, and body.
                var mergedModel = new MergedModel
                {
```

```
                    Header = headerValue,  // Setting the Header property with
the value obtained from the request header.
                    Query = queryValue,    // Setting the Query property with the
value obtained from the request query string.
                    BodyData = bodyModel?.Data  // Setting the BodyData property
with the "Data" from the deserialized body content, using null-conditional
operator to handle potential null values.
                };

                // Marking the model binding as successful and passing the merged
model back to the framework.
                bindingContext.Result = ModelBindingResult.Success(mergedModel);
            }
            catch (Exception ex)
            {
                // General error handling: Catch any unexpected exceptions and
report failure.
                bindingContext.ModelState.AddModelError("MultiSourceModelBinder",
$"An unexpected error occurred: {ex.Message}");
                bindingContext.Result = ModelBindingResult.Failed();  // Mark the
binding as failed.
            }
        }
    }
}
```

Modifying the MultiSourceBinding Action Method:

Next, please modify the MultiSourceBinding Action Method as follows. Here, the **[ModelBinder(BinderType = typeof(MultiSourceModelBinder))]** attribute explicitly specifies that the custom model binder (MultiSourceModelBinder) should be applied to bind the MergedModel parameter in the MultiSourceBinding action method.

```
// The [ModelBinder] attribute is applied to the 'model' parameter, indicating
that a custom model binder will be used to bind this parameter.
// 'BinderType = typeof(MultiSourceModelBinder)' specifies that the
'MultiSourceModelBinder' class will handle the binding logic for this parameter.
[HttpPost("multi-source-binding")]
public IActionResult MultiSourceBinding([ModelBinder(BinderType =
typeof(MultiSourceModelBinder))] MergedModel model)
{
    // The action method returns an HTTP 200 OK response with the bound
'MergedModel' as the response body.
    // The 'MergedModel' object is populated using the custom binding logic
defined in the 'MultiSourceModelBinder'.
    return Ok(model);
```

```
}
```

With the above changes in place, access the **api/sample/multi-source-binding** endpoint using a **POST** request with the following details.

Headers: X-Custom-Header: Set this to HeaderValue.

Query Parameters: queryValue: Test.

Body (JSON):

{

“data”: “BodyValue”

}

Custom Model Binder for Special Data Types (Tuple Binding)

We will create a Custom Model Binder to handle tuples in the request body. So, create a class file named **TupleModelBinder.cs** within the Models folder and copy and paste the following code. The following custom model binder class binds the incoming HTTP request body (JSON format) to a Tuple<int, string> model. This custom binder reads the request body asynchronously, deserializes it into a Tuple<int, string>, and binds it to the corresponding model within the application.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;
using Newtonsoft.Json;

namespace ModelBinding.Models
{
    // Custom model binder class implementing the IModelBinder interface
    public class TupleModelBinder : IModelBinder
    {
        // Asynchronous method that binds the incoming request body to a
Tuple<int, string>
        public async Task BindModelAsync(ModelBindingContext bindingContext)
        {
            // Ensure that bindingContext is not null to avoid
NullReferenceException
            if (bindingContext == null)
            {
                throw new ArgumentNullException(nameof(bindingContext));
            }

            try
            {
                // Read the request body asynchronously and convert it to a
string
                var body = await new
StreamReader(bindingContext.HttpContext.Request.Body).ReadToEndAsync();
```

```
                // Check if the request body is empty
                if (string.IsNullOrEmpty(body))
                {
                    // Set the result to failure and add an error message if the
body is empty and exit the method

bindingContext.ModelState.AddModelError(bindingContext.ModelName, "Request body is
empty.");
                    bindingContext.Result = ModelBindingResult.Failed();
                    return;
                }

                // Deserialize the JSON body into a Tuple<int, string>
                var tupleData = JsonConvert.DeserializeObject<Tuple<int, string>>
(body);

                // Check if deserialization succeeded, if null, return failed
model binding and exit the method
                if (tupleData == null)
                {

bindingContext.ModelState.AddModelError(bindingContext.ModelName, "Invalid JSON
format or type mismatch.");
                    bindingContext.Result = ModelBindingResult.Failed();
                    return;
                }

                // Set the binding result as success if deserialization is
successful
                bindingContext.Result = ModelBindingResult.Success(tupleData);
            }
            catch (JsonException jsonEx)
            {
                // Handle JSON-specific errors (e.g., invalid JSON)
                bindingContext.ModelState.AddModelError(bindingContext.ModelName,
$"JSON deserialization error: {jsonEx.Message}");
                bindingContext.Result = ModelBindingResult.Failed();
            }
            catch (Exception ex)
            {
                // Handle general exceptions
                bindingContext.ModelState.AddModelError(bindingContext.ModelName,
$"An error occurred: {ex.Message}");
                bindingContext.Result = ModelBindingResult.Failed();
            }
        }
```

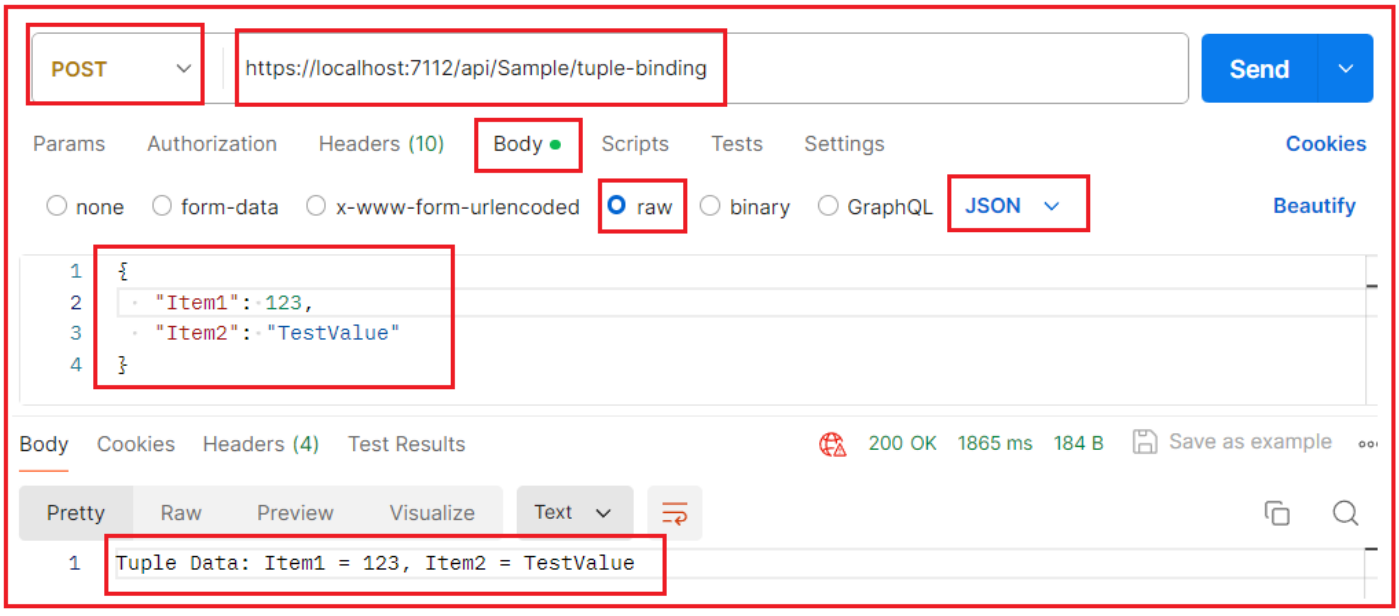
```
    }
}
```

Using the Custom TupleModelBinder with the Action Method:

Please modify the TupleBinding action method as follows. The use of the **[ModelBinder(BinderType = typeof(TupleModelBinder))]** attribute in the following code allows you to specify a custom model binder for binding the incoming request data to the Tuple<int, string> model parameter (tupleData) in the TupleBinding action method.

```
[HttpPost("tuple-binding")]
//The [ModelBinder] attribute with the BinderType property specifies that a custom model binder should be used instead of the default model binder.
//In this case, BinderType = typeof(TupleModelBinder) indicates that the TupleModelBinder class should be used to bind the incoming request data to the Tuple<int, string> parameter(tupleData).
public IActionResult TupleBinding([ModelBinder(BinderType = typeof(TupleModelBinder))] Tuple<int, string> tupleData)
{
    return Ok($"Tuple Data: Item1 = {tupleData.Item1}, Item2 = {tupleData.Item2}");
}
```

With the above changes in place, access the above endpoint (api/Sample/tuple-binding). It should work as expected, as shown in the image below. I am using Postman to access the above endpoint.



Custom Model Binder for Handling Large Data Efficiently

We will create a custom binder that processes large datasets more efficiently using streaming instead of reading the entire payload into memory at once. So, create a class file named **LargeDataModelBinder.cs** within the Models folder and copy and paste the following code. The following custom model binder called class binds large amounts

of data (line-by-line) from the HTTP request body to a LargeDataModel object. This custom model binder reads each line of the request body asynchronously, stores it in a list, and checks if the list is empty before returning a success or failure result.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;

namespace ModelBinding.Models
{
    // Custom model binder class implementing the IModelBinder interface
    public class LargeDataModelBinder : IModelBinder
    {
        // Asynchronous method that binds the incoming request body to a
LargeDataModel object
        public async Task BindModelAsync(ModelBindingContext bindingContext)
        {
            // Ensure that bindingContext is not null to avoid
NullReferenceException
            if (bindingContext == null)
            {
                // Throws an exception if the binding context is null, ensuring
no null reference issues
                throw new ArgumentNullException(nameof(bindingContext));
            }

            try
            {
                // Initialize an empty list to hold the lines of large data from
the request body
                var largeDataList = new List<string>();

                // Create a StreamReader to read the request body content line by
line
                using (var reader = new
StreamReader(bindingContext.HttpContext.Request.Body))
                {
                    // Initialize a string variable to hold each line of the data
                    string? line;

                    // Read the request body line by line asynchronously
                    while ((line = await reader.ReadLineAsync()) != null)
                    {
                        // Add each non-null line to the list of large data
                        largeDataList.Add(line);
                    }
                }
```

```
                // After reading the entire body, check if the list is still
empty
                if (largeDataList.Count == 0)
                {
                    // If the list is empty, add an error to ModelState
indicating no valid data was found

bindingContext.ModelState.AddModelError(bindingContext.ModelName, "Request body is
empty or contains no valid data.");

                    // Mark the binding as failed
                    bindingContext.Result = ModelBindingResult.Failed();

                    // Exit the method since binding has failed
                    return;
                }

                // If data was successfully read and the list is not empty, set
the binding result as success
                bindingContext.Result = ModelBindingResult.Success(new
LargeDataModel { LargeDataList = largeDataList });
            }
            catch (Exception ex)
            {
                // Handle any general exceptions that occur during model binding
                // Add the exception message to ModelState to give details of
what went wrong
                bindingContext.ModelState.AddModelError(bindingContext.ModelName,
$"An unexpected error occurred: {ex.Message}");

                // Mark the binding result as failed due to the exception
                bindingContext.Result = ModelBindingResult.Failed();
            }
        }
    }
}
```

Using the Custom LargeDataModelBinder with the Action Method:

Please modify the LargeDataBinding action method as follows. The use of the **[ModelBinder(BinderType = typeof(LargeDataModelBinder))]** attribute in the following code specifies that a custom model binder for binding the incoming request data to the LargeDataModel model parameter in the LargeDataBinding action method.

```
[HttpPost("large-data-binding")]
public IActionResult LargeDataBinding([ModelBinder(BinderType =
typeof(LargeDataModelBinder))] LargeDataModel model)
```

```
{
    return Ok("Data received successfully");
}
```

With the above changes, access the endpoint (**api/Sample/large-data-binding**). It should work as expected, as shown in the image below. I am using Postman to access the above endpoint.

When Should We Create a Custom Model Binder in ASP.NET Core Web API?

A custom model binder in ASP.NET Core Web API is necessary when the default model binding behavior does not satisfy your application's needs. Custom model binders allow you to handle more complex scenarios and bind data

in ways the default binder cannot. The following are some of the scenarios where we can create a Custom Model Binding in ASP.NET Core:

- **Binding from Custom Data Formats:** If your input data follows a non-standard format that the default model binder cannot handle (e.g., complex strings, custom delimiters, or data in a special format like Name:Age:Location), a custom model binder is required to parse and bind this data to your model.
- **Binding from Multiple Sources:** If you need to bind data from multiple sources (e.g., combining data from headers, query strings, and the body into a single model), the default model binder won't suffice. A custom model binder allows you to extract and merge data from multiple parts of the request.
- **Special Handling for Non-Standard Data Types:** The default model binder struggles with non-standard data types such as tuples, dictionaries with complex keys/values, or custom collections. A custom model binder lets you define how these types are bound.
- **Handling Large Data Efficiently:** For large payloads, the default model binder (e.g., FromBody) reads the entire request body into memory, which can be inefficient. A custom model binder allows for more efficient handling of large data by streaming the data and avoiding loading everything into memory at once.

In the next article, I will discuss **How to Apply Binding Attributes to Model Properties in ASP.NET Core Web API** with Examples. In this article, I try to explain **Custom Model Binding in ASP.NET Core Web API** with Examples. I hope you enjoy this article, "Custom Model Binding Using FromBody in ASP.NET Core Web API."

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.